

Entrenar redes neuronales cuánticas para hardware limitado por T1: qué hace y qué no hace el filtro de simetría qg

Vicente Humberto Monteverde

Universidad del Museo Social Argentino (UMSA), Argentina

ORCID 0000-0001-8884-4811

Borrador del 5 de octubre de 2026

Resumen

Una red neuronal cuántica (QNN) construida con compuertas que conservan el peso de Hamming mantiene sus datos en un sector de peso, así que los disparos que salieron del sector pueden descartarse en la lectura. Este es el filtro qg de la librería qang [1, 2]. Mostramos que, con igual amortiguamiento de amplitud (T1) en todos los qubits, la lectura filtrada de una red así es exactamente la lectura sin ruido, en cualquier sector de peso. La fracción de disparos conservados es $(1 - \gamma)^{k_d}$ para peso k y profundidad d , y entrenar bajo T1 con el filtro da exactamente los parámetros del entrenamiento sin ruido. Luego medimos cuánto vale esto en cuatro conjuntos de clasificación pequeños. Cada resultado se reporta con el filtro y sin él, y cada predicción quedó registrada antes de su corrida, fallas incluidas. En 60 corridas por modelo, una red entrenada en simulador y ejecutada bajo T1 gana +2.5 puntos con el filtro en peso 1 (IC 95 % [0.8, 4.1]) y +16.5 puntos en peso 2 ([11.8, 21.2]), y recupera exactamente su precisión sin ruido. Una red entrenada con el ruido calibrado sin el filtro la alcanza (diferencias de 0.1 puntos), y el filtro no corrige ni el desfase ni un T1 distinto entre qubits, aunque solo pierde menos de un punto hasta una dispersión de $\pm 80\%$ en $1/T_1$. Con entrenamiento con ruido y una lectura más rica, el modelo sin filtro fue 1.2 puntos mejor en un estudio, pero en 60 corridas nuevas esa diferencia bajó a +0.5 puntos y no fue significativa. El déficit del peso 2 viene de la codificación por productos de pares, no de la lectura; con una codificación que llena todo el sector y una lectura de rango completo, el peso 2 alcanza la precisión del peso 1. Con los modelos de ruido de equipos de IBM e IonQ, el filtro reduce el error del qgz medido entre 2.7 y 3.6 veces; con el modelo de ruido Forte-1 de IonQ, en 189 entradas de cuatro conjuntos, el modelo filtrado es 1.1 puntos más preciso que el crudo, y calibrar con cinco circuitos de eco por modelo los errores que deja el filtro reduce a la mitad el error restante del valor de decisión: las decisiones que difieren del modelo sin ruido bajan de 15 (crudo) y 7 (filtro) a 3 en 945 lecturas sobre cinco modelos de ruido de equipos. Con una codificación fiel de peso 2 desaparece la ventaja del modelo sin filtro entrenado con el ruido, y un clasificador que recibe a la vez las variables filtradas y las crudas no gana nada sobre el filtro solo. En 513 configuraciones de lectura, incluidas fracciones conservadas de hasta 2.6×10^{-7} , T1 desigual y desfase, el filtro tiene el menor error de lectura siempre que se conserven unos 20 disparos, y una regla cerrada predice el ganador. Lo mismo vale más allá del primer modelo: con 8 qubits una red de peso 2 mantiene su 0.965 sin ruido con el filtro y cae a 0.705 sin él; un clasificador de cinco clases (una clase por qubit) mantiene su precisión sin ruido bajo T1 con el filtro y pierde entre 15 y 18 puntos sin él en dígitos de cinco clases; entrenar con 100 o 1000 disparos por paso con el filtro queda entre 1.0 y 1.5 puntos por delante de entrenar sin él; y el gradiente filtrado es igual al sin ruido, mientras que el crudo es K veces menor en peso 1. En cinco modelos de ruido de equipos la calibración por eco revierte confusiones de clase: las decisiones cambiadas del modelo de cinco clases bajan de 51 (crudo) y 24 (filtro) a 15 en 500 lecturas. El eco ayudó solo donde el error filtrado estaba bien por encima del ruido de disparo. Estos modelos son simulables clásicamente y no se afirma ventaja cuántica. El filtro es una manera de hacer válido el entrenamiento sin ruido bajo relajación, con un costo en disparos conocido de antemano. Los modelos se publican

como `qang.qml` (versión 0.6.16, `pip install qang`; el modelo multiclase está en la rama principal del repositorio).

1 Introducción

Los clasificadores cuánticos variacionales se entrenan en simuladores y se ejecutan en equipos con ruido, o se entrenan directamente con el ruido del equipo. El entrenamiento consciente del ruido [3] es el remedio habitual cuando ambos difieren. La verificación de simetría [4, 5] es otro remedio: cuando el circuito ideal conserva una cantidad, se descartan los resultados que la violan. El proyecto qang expresa las lecturas de qubits en unidades qg ($qg_Z = \langle \sigma_z \rangle$ y sus compañeras) [1]. Construyó un testigo de media de registro y un filtro por disparo para circuitos que conservan un peso de Hamming [2]. Esta nota hace una pregunta acotada: para una QNN que conserva el peso, ¿cuánto cambia ese filtro la precisión de un clasificador en hardware limitado por T1, y cuándo deja de importar?

Respondemos con simulación exacta, un protocolo fijo y una comparación explícita. Cada lectura se toma con qang (filtrada) y sin qang (cruda), sobre los mismos parámetros entrenados o la misma inicialización. Las predicciones de cada estudio quedaron registradas en el repositorio antes de la corrida que las puso a prueba. De 122 predicciones pre-registradas en los estudios que se resumen aquí, 89 se cumplieron y 33 fallaron; las fallas se reportan junto a los aciertos.

2 Modelo y teoría

QNN que conserva el peso. En n qubits, una compuerta RBS (*reconfigurable beam splitter*) actúa sobre un par (a, b) como una rotación de Givens entre $|10\rangle$ y $|01\rangle$ y deja fijos $|00\rangle$ y $|11\rangle$, así que conserva el peso de Hamming [6]. Usamos $n = 5$ y tres capas de compuertas RBS sobre los pares $\{(0, 1), (2, 3)\}$, $\{(1, 2), (3, 4)\}$, $\{(4, 0)\}$. Eso da 15 ángulos y profundidad $d = 9$ (subcapas). La lectura es $z = \sum_i c_i qg_Z^{(i)} + b$ con c_i, b entrenables y pérdida logística.

Codificaciones. Con cuatro variables $x \in [-1, 1]^4$ sea $v = (x, 1)/\|(x, 1)\|$; la componente constante conserva $\|x\|$. El modelo E (peso 1) pone v_i en el estado base con el qubit i excitado. El modelo W (peso 2) pone $v_i v_j$ en el estado con los qubits $i < j$ excitados, normalizado. Los sectores tienen dimensión 5 y 10.

El filtro. Con qang, la lectura conserva solo los disparos cuyo peso es igual al peso de entrada k y renormaliza (`qang.sectors.filter_distribution`). Sin qang, se usan todos los disparos.

F1/F4: exactitud con T1 igual. Sea un amortiguamiento de amplitud con probabilidad γ en cada qubit después de cada subcapa. Su operador de Kraus sin salto multiplica un estado base con k excitaciones por $(1 - \gamma)^{k/2}$: el mismo factor para todos los estados del sector de peso k . Todo salto baja el peso y sale del sector. Las compuertas que conservan el peso nunca devuelven un estado rebajado. Por eso, condicionado a que el peso sea k , el estado es exactamente el sin ruido, y

$$\text{fracción conservada} = (1 - \gamma)^{kd}. \quad (1)$$

Para $\gamma = 0.08$ y $d = 9$ es 0.472 en peso 1 y 0.223 en peso 2. Si el amortiguamiento cambia entre qubits, el factor sin salto es $\prod_{q \in S} (1 - \gamma_q)^{1/2}$ sobre el conjunto excitado S , que no es uniforme, y se pierde la exactitud. El desfase conserva el peso, así que el filtro no lo ve.

F3: entrenamiento. Como la lectura filtrada bajo T1 es igual a la sin ruido para cualquier valor de los parámetros, la pérdida y todos los gradientes son idénticos. Entrenar bajo T1 con el filtro da los parámetros del entrenamiento sin ruido, con la misma inicialización.

Lectura multiclasa. MultiClassQNN usa la red de peso 1 con $C \leq n$ clases, la clase c en el qubit c . Su lectura “qubit” puntúa la clase c con la probabilidad de excitación $p_c = (1 - qg_Z^{(c)})/2$ y logits $a p_c + b_c$; su lectura “cabeza” es un softmax lineal sobre los n valores $qg_Z^{(i)}$. Como cada p_c es una probabilidad del sector, vale F4: con T1 igual las puntuaciones filtradas son exactamente las sin ruido. Un error dentro del sector mueve aquí la excitación a otro qubit, es decir, cambia una clase por otra.

F2: simulabilidad. En peso 1 la red es una matriz ortogonal O de 5×5 sobre v , y z es una forma cuadrática en v . En peso k el sector tiene dimensión $\binom{n}{k}$, polinomial para k fijo. Estos modelos se pueden simular clásicamente. Si versiones más grandes y de mayor peso dejarían de serlo es otra pregunta [7] que esta nota no aborda.

3 Métodos

Conjuntos de datos (binarios): iris (versicolor contra virginica), cáncer de mama, vino (clases 0 contra 1) y dígitos (3 contra 8), con 100 muestras por clase cuando hay. Cada corrida usa una partición estratificada 70/30, estandarización, PCA a cuatro componentes donde hace falta y escalado a $[-1, 1]$; una muestra de prueba vale 2–3 puntos. El entrenamiento usa Adam durante 120 épocas, con gradientes por diferencias centrales en los ángulos. Las condiciones de ruido se aplican después de cada subcapa: T1 igual con $\gamma = 0.08$; T1 desigual con γ_q de 0.04 a 0.12 (media 0.08); y T1 igual más un cambio de fase con probabilidad 0.03. La simulación es exacta. T1 y desfase mantienen la matriz densidad diagonal por bloques según el peso de Hamming, así que se propaga un bloque por peso; se verificó contra la matriz densidad completa a 10^{-12} . Protocolo: el script de cada estudio, con sus predicciones escritas en el encabezado, se registró antes de usar la semilla de su corrida reportada; el código se depuró con otra semilla y pocas épocas. Los hashes de los commits están en las notas de investigación [1].

4 Resultados

Estudios con una semilla. La QNN que conserva el peso y la norma llega a una precisión media de 0.960 sobre los cuatro conjuntos. Queda a la par de una QNN estándar de cuatro qubits que no conserva el peso (0.952), de una máquina de vectores soporte RBF (0.957) y de una red MLP pequeña (0.961). Sin la componente de norma, la misma red obtiene 0.791. Entrenada sin ruido y ejecutada bajo T1 con el filtro, conserva exactamente su precisión sin ruido. Entrenar bajo T1 con el filtro reproduce el entrenamiento sin ruido a 5×10^{-9} en los parámetros (F3). Con $\gamma = 0.08$, una QNN estándar entrenada sin ruido colapsa bajo T1 (0.771); entrenada bajo T1 se recupera (0.944).

Las predicciones que fallaron:

- la codificación arccos no mejoró la precisión;
- el gemelo cuadrático clásico no quedó a menos de 1 punto de la QNN con la regularización por defecto;
- el T1 desigual le costó al filtro 1.5 puntos en una corrida (límite: 1 punto);
- el peso 2 fue menos preciso que el peso 1 (por 3.7 puntos);
- entrenado bajo T1, el modelo de peso 2 sin filtro quedó 0.6 puntos por delante del filtrado;
- en los estudios de la Sec. 4: la lectura $\langle Z_i Z_j \rangle$ no ayudó al peso 2 (dos predicciones), el filtro no fue neutral con entrenamiento con ruido y esa lectura, la ganancia del filtro en una dispersión de T1 tuvo un IC que tocó el cero, el peso 2 no fue más sensible a la dispersión

que el peso 1, la codificación por productos de pares tuvo un techo de información menor que la unaria, y llenar todo el sector de peso 2 no ayudó con la lectura *qgz*;

- en la Sec. 4: con menos de un disparo conservado el filtro no perdió, al revés de lo previsto; el error crudo del déficit de radio no fue menor a mitad de llenado; y tres de las cuatro predicciones de la lectura de dos canales, incluida la réplica del efecto de 1.2 puntos; en modelos de ruido de equipos, el filtro quitó menos de la mitad del déficit de radio falso de los estados producto; y tres de cinco predicciones sobre codificaciones que distorsionan, incluida una segunda réplica del efecto de 1.2 puntos;
- en la Sec. 4: las cuatro predicciones de que el eco ayudaría en entradas de margen estrecho; el entrenamiento con 1000 disparos, donde se esperaban las dos lecturas a menos de 1 punto (*qang* +1.0), y con 100 disparos, donde se esperaba que el filtro perdiera más de un punto por su costo en disparos (quedó 1.5 puntos por delante); la pérdida de 10 puntos de la lectura cabeza multiclase sin filtro (9.1 puntos); una pérdida sin filtro mayor con 8 qubits que con 5 en peso 1 (se mantuvo en 3.1–3.4 puntos); un gradiente crudo suprimido más que K a mitad de llenado, y menos disparos con el filtro en todas las celdas de gradiente (10 de 12); y, en el eco multiclase, tantas decisiones coincidentes con el eco en cada backend y una lectura cabeza filtrada al menos tan precisa como la cruda en cada backend (cada una falló en uno o dos backends por una entrada).

Réplica en 60 corridas. Las comparaciones clave se repitieron con 5 semillas $\times$ 3 particiones $\times$ 4 conjuntos. La Tabla 1 y la Fig. 1 dan los resultados; las cinco predicciones pre-registradas de esta réplica se cumplieron.

lectura	E (peso 1)		W (peso 2)	
	con <i>qang</i>	sin <i>qang</i>	con <i>qang</i>	sin <i>qang</i>
exacta (sin ruido)	0.952		0.922	
T1, entrenada sin ruido	0.952	0.928	0.922	0.757
T1, entrenada con el ruido	0.952	0.951	0.922	0.924
T1 desigual, entrenada sin ruido	0.947	0.921	0.916	0.732
desfase, entrenada sin ruido	0.918	0.792	0.882	0.656
desfase, entrenada con el ruido	0.950		0.926	

Tabla 1: Precisión de prueba media en 60 corridas por modelo, con y sin el filtro *qg*.

Las diferencias pareadas (media, IC 95 % entre semillas) fueron las siguientes.

- *Ganancia del filtro, entrenada sin ruido, bajo T1*: +2.5 [0.8, 4.1] puntos (E) y +16.5 [11.8, 21.2] (W). El modelo filtrado quedó adelante o empatado en 50/60 (E) y 56/60 (W) corridas.
- *Con entrenamiento con ruido*: +0.1 [−0.6, 0.9] (E) y −0.1 [−0.8, 0.5] (W).
- *Pérdida por T1 desigual*: 0.5 [−0.2, 1.2] (E) y 0.7 [0.0, 1.3] (W). Los 1.5 puntos de la corrida anterior estaban en el extremo alto de la dispersión.
- *Desfase, entrenamiento con ruido menos sin ruido, con filtro*: +3.2 (E) y +4.5 (W), ambos IC por encima de 0.
- *Con desfase, un modelo entrenado sin ruido igual gana* +12.6 (E) y +22.6 (W) puntos con el filtro, que elimina la parte T1 del ruido combinado.
- *Peso 2 frente a peso 1*: −3.0 [−4.0, −2.0] puntos.

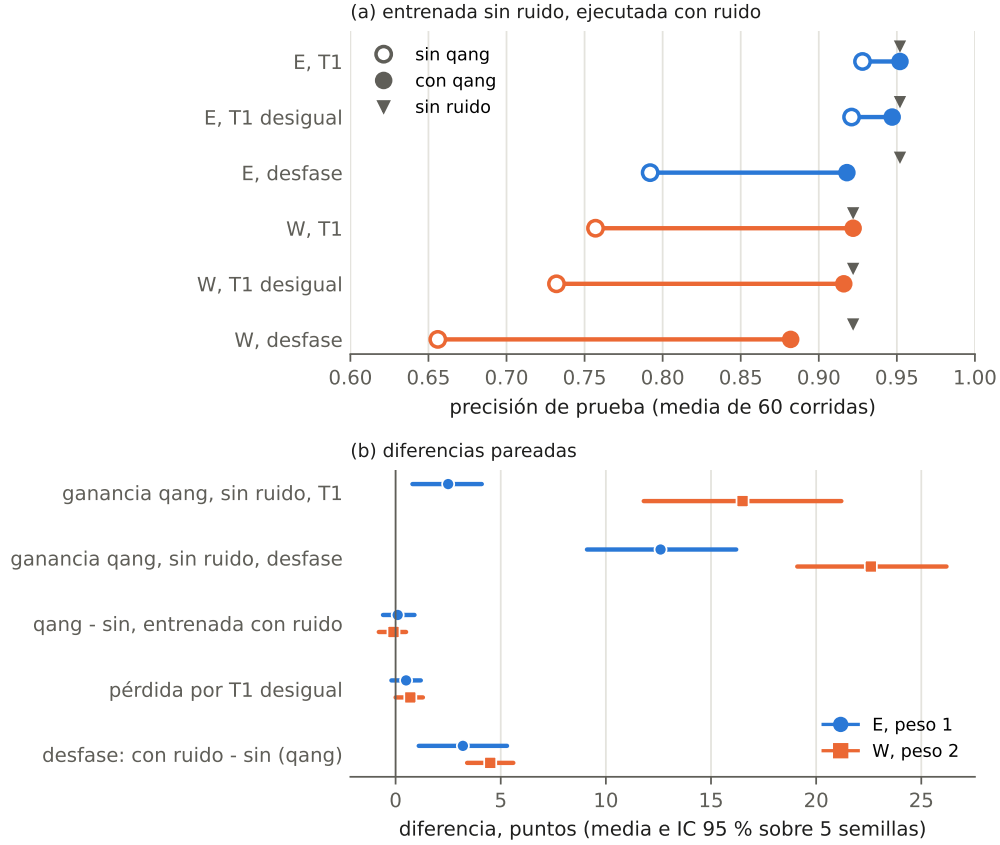


Figura 1: (a) Precisión de redes entrenadas sin ruido y ejecutadas con ruido, sin (vacío) y con (lleno) el filtro; los triángulos marcan la precisión sin ruido. (b) Diferencias pareadas, media e IC 95 % sobre 5 semillas.

Lectura, codificación y dispersión de T1 (60 corridas cada una).

- *Lectura de correlaciones.* Agregar las $\binom{5}{2} = 10$ correlaciones $\langle Z_i Z_j \rangle$ hace que la lectura de peso 2 sea una función lineal de toda la distribución del sector (rango 10 en lugar de 5). No ayudó: -0.6 $[-1.5, 0.4]$ puntos. Entrenado bajo T1 con 15 pesos de lectura, el modelo *sin* filtro fue 1.2 $[0.4, 2.0]$ puntos mejor. Los disparos decaídos todavía informan dónde estaban las excitaciones, y el filtro descarta esa información. El filtro sigue siendo exacto para esta lectura, y entrenando sin ruido suma $+14.9$ puntos.
- *Codificación.* La mejor forma cuadrática sin restricciones en las amplitudes de productos de pares llega a 0.928, 2.4 $[1.7, 3.0]$ puntos por debajo del mismo techo para las amplitudes unarias (0.952). La normalización de los productos distorsiona los datos. Cargar los datos de peso 1 en cinco estados de peso 2 (un “anillo” de pares $(k, k+1)$) da 0.943, a 1.0 punto del peso 1 y 2.3 $[0.7, 3.9]$ puntos por encima de los pares. Con esta codificación la ganancia del filtro bajo T1 es la mayor vista, $+23.9$ puntos. Una codificación “dual” llena los diez estados de peso 2 sin distorsión (v en el anillo, $(x^2, 1)/\|(x^2, 1)\|$ en las cuerdas $(k, k+2)$). Con la lectura *qgz* no ayuda (-0.2 puntos frente al anillo), pero con la lectura $\langle Z_i Z_j \rangle$ de rango completo el modelo de peso 2 alcanza la precisión del peso 1 (0.947 contra 0.947 en 60 corridas). El peso 2 necesitaba una codificación fiel y una lectura que vea todo el sector.
- *Dispersión de T1.* Con $\gamma_q = 0.08(1 + s u_q)$ y u_q repartido en $[-1, 1]$, la pérdida del filtro fue de 0.3/0.2 puntos (peso 1/2) con $s = 0.5$ y 1.4/1.3 con $s = 1$. El filtro solo mantiene la pérdida por debajo de un punto hasta $s \approx 0.8$, es decir, una dispersión de $\pm 80\%$ de las

tasas $1/T_1$ alrededor de su media.

Modelos de ruido de equipos. El modelo de peso 1 se compiló a Qiskit: una cascada de 4 compuertas RBS carga las amplitudes unarias con signo, y siguen las 15 RBS entrenadas; el circuito coincide con el modelo a 10^{-16} . Se corrió con 30 entradas de prueba de iris y 1000 disparos cada una, leídas con y sin filtro de los mismos disparos. Los modelos de ruido fueron los de tres equipos de IBM (backends simulados con su calibración, en Aer) y el simulador en la nube de IonQ con los modelos de ruido Aria-1 y Forte-1. En los cinco la precisión fue 0.933 con y sin filtro, igual a la precisión sin ruido: los márgenes de este modelo son amplios. El error del qgz medido bajó de 0.138–0.207 sin filtro a 0.040–0.069 con filtro (entre 2.9 y 3.6 veces), conservando el 69–74 % de los disparos. Con el modelo de ruido Forte-1 de IonQ se corrió el mismo proceso en los cuatro conjuntos (189 entradas de prueba): precisión conjunta 0.968 con filtro, 0.958 sin filtro y 0.979 sin ruido, y un error de qgz entre 2.7 y 2.9 veces menor con filtro en cada conjunto. (Los resultados de IonQ usan las frecuencias medidas que devuelve la API; `get_counts` de qiskit-ionq las vuelve a muestrear en cada llamada.) Son modelos de ruido, no equipos; las corridas en hardware están preparadas, con confirmación de costo, y pendientes.

Calibración por eco de los errores que deja el filtro. El filtro descarta los disparos que salieron del sector, pero los errores que mueven una excitación dentro del sector pasan: en los modelos de ruido de equipos, entre el 8 y el 23 % de los disparos conservados de un estado producto ponen la excitación en el qubit equivocado. Un circuito de eco (qubit j excitado, las 15 compuertas RBS entrenadas y su inversa) debería devolver el qubit j ; su distribución filtrada es la columna j de una matriz de transferencia M de 5×5 , e invertir $M^{1/2}$ sobre la distribución filtrada de cada entrada mitiga esos errores, como hace la mitigación de lectura con los errores de bit (`qang.sectors.echo_transfer_matrix`, `unmix_sector`). Con 5 circuitos de eco por modelo entrenado, en las 189 entradas de los cuatro conjuntos, los tres modelos de ruido de IBM y el simulador de IonQ en su conjunto de compuertas nativo (el compilador de IonQ elimina identidades de los circuitos en su conjunto estándar, así que los ecos deben enviarse en nativo):

- el error del valor de decisión respecto del modelo sin ruido bajó entre un 43 y un 56 % respecto del filtro solo, y el filtro más el eco quedó entre 3.7 y 4.6 veces por debajo de la lectura cruda;
- en 945 lecturas, 15 decisiones difirieron del modelo sin ruido sin el filtro, 7 con el filtro y 3 con el filtro más la calibración por eco; en los tres modelos de ruido de IBM, filtro más eco alcanzó la precisión sin ruido (0.979);
- las doce predicciones de estos tres estudios se cumplieron.

Dos estudios posteriores marcan los límites del eco. En el sector de peso 2 (una matriz de 10×10 con 10 ecos, 30 entradas, los mismos cinco backends) quitó entre el 37 y el 43 % del error de qgz que deja el filtro y entre el 39 y el 48 % del error de $\langle Z_i Z_j \rangle$, y las cuatro predicciones se cumplieron. En 120 entradas de margen estrecho de un modelo de peso 1, elegidas antes de las corridas con ruido, las cuatro predicciones fallaron: el filtro bajó las decisiones cambiadas de 47 a 17 en 600 lecturas, y el eco dejó 19. Ahí el error filtrado (0.26–0.35) estaba cerca de la dispersión por ruido de disparo del valor de decisión (unos 0.22), así que quedaba poco sesgo que quitar e invertir la matriz sumó varianza. El eco vale sus circuitos cuando el error filtrado está bien por encima de su nivel de ruido de disparo, que muestra una simulación sin ruido con los mismos disparos.

Dos canales, costo en disparos y qué más lee el filtro.

- *Lectura de dos canales (60 corridas)*. Darle al clasificador las variables filtradas y las crudas (2×15 pesos, `qang="both"`) buscaba conservar la información de los disparos decaídos que hizo al modelo crudo 1.2 puntos mejor. Con la codificación dual ese efecto no se repitió: entrenado bajo T1, el modelo filtrado iguala al sin ruido (0.953) y el crudo llega a 0.950 (-0.3 [$-0.8, 0.2$] puntos contra el filtro). Los dos canales igualan a la lectura cruda (-0.0 [$-0.6, 0.6$]) y, con peso 1, a la mejor lectura sola ($+0.0$ [$-0.4, 0.4$]).
- *Codificación que distorsiona contra codificación fiel (60 corridas nuevas)*. Entrenado bajo T1 con la codificación por productos de pares y la lectura de 15 pesos, el modelo crudo quedó $+0.5$ [$-0.7, 1.6$] puntos adelante, no 1.2; con la codificación dual quedó -0.4 [$-1.2, 0.4$] atrás, y la diferencia entre ambas ($+0.9$ [$-0.6, 2.4$]) no fue significativa. Con la lectura *qgz* de 5 pesos el filtro fue 0.6 [$0.1, 1.0$] puntos mejor. El único caso en que el filtro perdía no se sostiene como efecto significativo.
- *Costo en disparos*. Para la lectura *qgz* de circuitos aleatorios que conservan el peso, el error filtrado sigue $(1 - z^2)/(KS)$ y el crudo $(r - z)^2 + (1 - r^2)/S$, con K la fracción conservada y S los disparos. En 108 configuraciones (4–8 qubits, peso 1–4, profundidad 3–24) el filtro tiene el menor error en 106 (hasta 1000 veces menor) y la regla acierta el ganador en el 99%; el equilibrio está entre 3 y 110 disparos. En otras 405 configuraciones, con K hasta 2.6×10^{-7} , T1 desigual y desfase, gana en todas las que conservan al menos 20 disparos (hasta 1871 veces con T1 igual, 94 con T1 desigual y 58 con desfase), y una regla generalizada con el valor filtrado sesgado acierta en 342 de 342 casos con $KS \geq 5$. Cuando sobreviven menos disparos, la lectura cruda también perdió la información, así que el filtro tampoco perdió.
- *Entrelazamiento después del filtro*. En un estado de peso fijo el radio de Bloch de cada qubit es $|qgz|$, y si el registro es puro $1 - qgz^2$ es su entrelazamiento con el resto. Con T1 igual el valor filtrado coincide con el sin ruido a 3×10^{-15} ; en estados producto es exactamente 0, mientras que la lectura cruda muestra un valor falso de 0.60–0.95. En cinco modelos de ruido de equipos (IBM e IonQ) el filtro reduce entre 3.2 y 4.0 veces el error de esta lectura en circuitos entrenados, pero los errores que mueven una excitación dentro del sector pasan el filtro y siguen haciendo parecer entrelazados a los estados producto.

Más allá del primer modelo: disparos, clases, tamaño y gradientes. La Tabla 2 reúne cinco estudios que cambian cada uno una cosa del planteo anterior.

- *Entrenar con disparos* (4 conjuntos $\times$ 5 semillas). El modelo E se entrenó bajo T1 con S disparos nuevos por evaluación (SPSA en los ángulos) y se probó con 1000 disparos. Con el filtro llegó a 0.939 ($S = 1000$) y 0.930 ($S = 100$), sin él a 0.929 y 0.915. El costo en disparos del filtro no se notó en el entrenamiento. Entrenar sin ruido y aplicar el filtro al ejecutar (0.945) fue tan bueno como cualquier otra vía.
- *Multiclase* (iris, wine, dígitos 0–4; 5 semillas). Con el filtro cada corrida iguala su precisión sin ruido. Sin él la lectura cabeza pierde 9.1 puntos en promedio y 18 en los dígitos de cinco clases; la lectura qubit pierde menos, porque con T1 igual los p_c crudos se encogen juntos (en iris, de tres clases, incluso quedó 0.9 puntos por delante). Entrenado con el ruido, el modelo crudo queda a menos de 2 puntos.
- *5, 6 y 8 qubits* (cancer, wine, dígitos; 3 semillas; profundidad 9). Las capas RBS se construyen directamente en cada sector de peso (`block_unitaries`), lo que hace 13 veces más rápido el entrenamiento con 8 qubits. Con el filtro cada corrida iguala su precisión sin ruido. Sin él la pérdida en peso 1 se mantiene en 3.1–3.4 puntos de 5 a 8 qubits; en peso 2 crece de 8.7 a 26.0 puntos. El entrenamiento con ruido sin filtro queda a menos de 0.9 puntos en cada celda.

- *Gradientes* (circuitos aleatorios, $n = 4, 6, 8$, peso 1 y $n/2$, 200 sorteos por celda, $\gamma = 0.02$). El gradiente filtrado es igual al sin ruido; la varianza del gradiente crudo es K^2 veces menor en peso 1, y se suprime menos que K a mitad de llenado, donde los disparos decaídos conservan parte de la señal. El propio sector aplanar el gradiente, con o sin filtro, 5.2 veces (peso 1) y 20 veces (mitad de llenado) de 4 a 8 qubits. Contando los disparos conservados, el filtro resolvió el gradiente con menos disparos en 10 de 12 celdas; con $n = 8$, $k = 4$ ($K \leq 0.14$) necesitó entre 1.3 y 2.2 veces más.
- *Multiclase en modelos de ruido de equipos* (dígitos 0–4, 100 entradas, 1000 disparos, los cinco backends de la Sec. 4, 5 circuitos de eco por modelo). Aquí un error dentro del sector es una confusión de clase, y el eco corrige una parte: en promedio sobre los backends la lectura cabeza dio 0.858 cruda, 0.890 con el filtro y 0.900 con filtro más eco, contra 0.91 sin ruido.

estudio	planteo	sin ruido	con qang	sin qang
disparos (T1)	entrenado con $S = 1000$ / $S = 100$	0.945	0.939 / 0.930	0.929 / 0.915
multiclase (T1)	lectura qubit / cabeza	0.918 / 0.953	0.918 / 0.953	0.865 / 0.863
tamaño (T1)	8 qubits, peso 1 / peso 2	0.977 / 0.965	0.977 / 0.965	0.946 / 0.705
gradientes (T1)	razón de varianza, peso 1	1	1	K^2
multiclase (equipos)	cabeza, cambios en 500	0.91	0.890 (24)	0.858 (51)
	con calibración por eco		0.900 (15)	

Tabla 2: Estudios más allá del primer modelo, con y sin el filtro (precisión media salvo indicación).

5 Discusión

La diferencia que hace el filtro depende de cómo se entrenó la red. Entrenada en simulador y ejecutada con relajación, el filtro es decisivo. Su efecto crece con el peso, porque un estado de peso k decae k veces más rápido. Entrenada con un modelo de ruido calibrado, una red sin filtro aprende el decaimiento, y el filtro no agrega nada medible. Lo que aporta el filtro es, por lo tanto, de procedimiento y no una ganancia de precisión sobre la mejor alternativa. Con T1 igual hace exactamente válido el entrenamiento sin ruido, sin modelo de ruido ni corridas de entrenamiento con ruido, y su costo (los disparos descartados) se conoce antes de ejecutar. El T1 desigual y el desfase no se corrigen. En hardware con mucha dispersión de T1 o desfase fuerte, el filtro complementa un modelo de ruido y no lo reemplaza. De los estudios anteriores salen dos reglas prácticas. Hasta una dispersión de $\pm 80\%$ de $1/T_1$ entre qubits, una red entrenada en simulador puede ejecutarse solo con el filtro, perdiendo menos de un punto. Si en cambio la red se entrena con el ruido calibrado, con una codificación que distorsiona y una lectura con muchos parámetros, los disparos decaídos parecieron útiles en un estudio (1.2 puntos), pero en 60 corridas nuevas la diferencia fue de 0.5 puntos y no significativa; con una codificación fiel no apareció, y sumar las variables crudas a las filtradas no ganó nada. Para modelos de peso 2, importa más cómo se cargan los datos que qué se lee. El costo en disparos del filtro no decidió en su contra en ninguno de nuestros mapas de escala. Lo que el filtro no ve, los errores que mantienen la excitación en el sector pero la mueven, se puede calibrar con circuitos de eco, al costo de cinco circuitos extra por modelo, siempre que el error filtrado esté bien por encima del ruido de disparo. Estas conclusiones se mantuvieron al entrenar con disparos, al pasar a cinco clases o al crecer a 8 qubits; con el tamaño, la pérdida sin filtro creció solo en peso 2. El filtro también le entrega al entrenamiento el gradiente sin ruido, pero no cura el aplanamiento que trae el propio sector con n mayor.

Limitaciones. Todo el ruido es simulado, incluidos los modelos de ruido de equipos. Hay una arquitectura por peso, 5–8 qubits y conjuntos pequeños en los que una muestra de prueba vale 2–3 puntos. El modelo de peso 2 alcanza pero no supera la precisión del peso 1 en estos conjuntos. No se afirma ventaja cuántica: estas redes son simulables clásicamente. Una corrida en hardware con el filtro es la siguiente prueba natural.

Disponibilidad de código y datos

Modelos: `qang.qml` en `qang 0.6.16` (`pip install qang`), solo NumPy, con `WeightQNN` (pesos 1 y 2, lecturas "z" y "zz", codificaciones "pairs", "ring" y "dual"; cualquier peso para simular; `qang="both"`), `kept_fraction` y `compare_qang`; `MultiClassQNN` y `block_unitaries` en la rama principal; calibración por eco en `qang.sectors` (`echo_transfer_matrix`, `unmix_sector`). Script listo para hardware: `examples/qnn_hardware_qg.py` (IBM e IonQ). Estudios: `examples/qnn*_qg.py` y `RESEARCH_NOTES` §75–§82, §85, §87, §90, §91 y §94–§101, §104–§113, con pruebas en `tests/`. Los cuadernos Colab `notebooks/qang_qml_es.ipynb` (español) y `qang_qml.ipynb` (inglés) instalan la librería desde PyPI (o desde GitHub para el modelo multiclase) y muestran cada resultado con y sin el filtro. Repositorio: <https://github.com/Viny2030/qang>.

```
from qang.qml import WeightQNN
m = WeightQNN(n_qubits=5, weight=2).fit(X_train, y_train)
m.score(X_test, y_test, gamma=0.08, qang=True) # = precision sin ruido
m.score(X_test, y_test, gamma=0.08, qang=False) # lectura cruda
```

Referencias

- [1] Vicente Humberto Monteverde. The Qang (qg): A unified angular-probability unit and metric for parametric quantum circuit design. 2026. doi: 10.5281/zenodo.22832150.
- [2] Vicente Humberto Monteverde. The register-mean polar bias as a symmetry witness: error mitigation, constrained optimization and qubit characterization in qg units. *Zenodo preprint*, 2026. doi: 10.5281/zenodo.23036963.
- [3] Hanrui Wang, Jiaqi Gu, Yongshan Ding, Zirui Li, Frederic T. Chong, David Z. Pan, and Song Han. QuantumNAT: quantum noise-aware training with noise injection, quantization and normalization. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2022. doi: 10.1145/3489517.3530400.
- [4] Xavier Bonet-Monroig, Ramiro Sagastizabal, M. Singh, and Thomas E. O’Brien. Low-cost error mitigation by symmetry verification. *Physical Review A*, 98:062339, 2018.
- [5] Sam McArdle, Xiao Yuan, and Simon Benjamin. Error-mitigated digital quantum simulation. *Physical Review Letters*, 122:180501, 2019.
- [6] Jonas Landman, Natansh Mathur, Yun Yvonna Li, Martin Strahm, Skander Kazdaghli, Anupam Prakash, and Iordanis Kerenidis. Quantum methods for neural networks and application to medical image classification. *Quantum*, 6:881, 2022. doi: 10.22331/q-2022-12-22-881.
- [7] M. Cerezo et al. Does provable absence of barren plateaus imply classical simulability? Or, why we need to rethink variational quantum computing, 2023. arXiv:2312.09121.